

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

Informe Técnico — Tercera Entrega del Proyecto Fin de Curso

Asistente Virtual Académico con Chatbot Híbrido

Aplicaciones Web — Quinto Nivel

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Equipo (Equipo H/CAFR):

Fajardo Montes Michael Xavier

Rivera Suárez Jonathan Javier

Castro Palma Justyn Moisés (se dio de baja durante el desarrollo)

Repositorio: github.com/Michael-XFM/asistente-academico

Etiqueta: v0.9.0-rc

Commit: [PENDIENTE — completar con el hash corto del último commit antes de entregar]

DOI (Zenodo): 10.5281/zenodo.21712828

Quevedo — Los Ríos — Ecuador

2026

Resumen ejecutivo

Este informe documenta el estado del proyecto "Asistente Virtual Académico con Chatbot Híbrido" al cierre de la Tercera Entrega (v0.9.0-rc), correspondiente a la asignatura Aplicaciones Web del quinto nivel de la carrera de Ingeniería de Software (UTEQ). El sistema es una API REST desarrollada en Spring Boot 3.5 / Java 21, con persistencia en PostgreSQL 16 y caché/blacklist de sesión en Redis 7, autenticación JWT, un frontend estático (HTML/CSS/JS), y una estrategia híbrida de acceso a datos que combina Spring Data JPA para operaciones CRUD elementales con funciones PL/pgSQL para operaciones agregadas.

Durante esta entrega se cerró el 100% de las observaciones de las Entregas 1A y 1B, se consolidó la ingeniería de requisitos (SRS, historias de usuario, casos de uso y matriz de trazabilidad), se verificó la reproducibilidad automática del sistema desde una clonación limpia (make up), se generó evidencia empírica cuantitativa en cuatro de los cinco sub-bloques exigidos (rendimiento, seguridad, cobertura de pruebas y calidad web/accesibilidad), y se documentó honestamente el estado de dos aspectos pendientes: la prueba de usabilidad SUS con participantes reales, y dos decisiones de arquitectura que se apartan de la especificación original de la guía (ver sección 2, "Estado del sistema").

Durante el trabajo de esta entrega se corrigieron además dos defectos funcionales reales encontrados durante la verificación: un control de acceso roto en el controlador de tareas (OWASP A01, ya corregido y probado con aislamiento explícito entre usuarios) y una expresión regular de validación de correo electrónico que rechazaba incorrectamente el propio dominio institucional (@uteq.edu.ec) por tener más de un punto en el dominio.

Introducción y alcance

El presente documento constituye el informe técnico exigido por el Bloque de Entregables de la guía de la Tercera Entrega del Proyecto Fin de Curso, asignatura Aplicaciones Web, quinto nivel de la carrera de Ingeniería de Software (UTEQ). Su propósito no es describir nuevas funcionalidades, sino consolidar y verificar, con evidencia empírica reproducible, el trabajo entregado hasta la Entrega 1B, elevando el proyecto al estado de release candidate v0.9.0-rc.

El alcance de este informe cubre los seis bloques exigidos por la guía: aplicación de observaciones (Bloque 0), consolidación funcional y estrategia de acceso a datos (Bloque A), reproducibilidad automática (Bloque B), evidencia empírica cuantitativa (Bloque C), documentación arquitectónica (Bloque D) y publicabilidad del artefacto (Bloque E). El documento se organiza siguiendo ese mismo orden, cerrando con una declaración de contribuciones, una declaración ética y las referencias bibliográficas utilizadas.

1. Observaciones de las Entregas 1A y 1B

La bitácora completa se encuentra en docs/observaciones/OBSERVACIONES.md. Resumen: 10 de 10 observaciones cerradas (100%). La resolución de cada observación es trazable a un commit específico del repositorio.

Código	Fuente	Criterio	Decisión (resumen)
OBS-01	1A	Arquitectura C4 N1	Se documentó la justificación de la pila tecnológica
OBS-02	1A	Modelo BD (crítico)	El DDL ajeno (clínica, MySQL) fue reemplazado por un esquema PostgreSQL real del dominio académico
OBS-03	1A	Wireframes	Fragmentos SQL ajenos eliminados; wireframes superados por pantallas HTML reales
OBS-04	1A	Repositorio	URL del repositorio verificada explícitamente
OBS-05	1B	Autenticación JWT	Se registró JwtAuthFilter en la cadena de seguridad; los tokens ahora se validan en cada petición
OBS-06	1B	CRUD / JPA	Blacklist de JTI en Redis para revocar tokens en logout
OBS-07	1B	CRUD / JPA	CRUD completo de la entidad Tarea expuesto vía API con paginación
OBS-08	1B	CRUD / JPA	Flyway incorporado con migraciones versionadas
OBS-09	1B	Seguridad OWASP	CORS restringido a orígenes explícitos (antes: comodín)
OBS-10	1B	Informe técnico	El informe técnico estructurado se entrega en esta Tercera Entrega (este documento)

2. Estado del sistema

2.1 Funcionalidad operativa

- Autenticación: registro, login con JWT firmado (HS256), logout con blacklist de token en Redis, límite de 5 intentos fallidos por IP (429 al sexto).
- CRUD completo de Tareas, con aislamiento estricto por usuario autenticado (404 si la tarea no existe, 403 si pertenece a otro usuario).
- Dashboard académico: resumen agregado (tareas pendientes, promedio, avisos) resuelto mediante funciones PL/pgSQL, no en el backend.
- Chatbot (HU-04 / CU-04): sin implementar en este ciclo; queda documentado como pendiente en la matriz de trazabilidad (REQ-F-006).

2.2 Limitaciones documentadas honestamente

Autenticación vía Bearer token, no cookie HttpOnly. La guía de la Tercera Entrega exige explícitamente JWT en cookie HttpOnly+Secure+SameSite=Strict (Bloque A.1). La implementación actual devuelve el token en el cuerpo JSON de la respuesta de login y lo recibe vía cabecera Authorization: Bearer en peticiones subsecuentes, patrón funcionalmente equivalente en seguridad de transporte (dado que todo el tráfico corre sobre HTTPS/TLS) pero que no corresponde literalmente al mecanismo exigido. Se recomienda documentar esta decisión en el ADR-002 o migrar a cookie HttpOnly antes de la Entrega Final.

Redis sin caché de respuestas HTTP. El Bloque A.1 exige un endpoint de listado cacheado con TTL externo y una tasa de aciertos (hit ratio) medida empíricamente. En el estado actual, Redis se usa exclusivamente para la blacklist de tokens JWT revocados (ver ADR-004), no para cachear respuestas de ningún endpoint. Esto se refleja también en el Bloque C.1 (rendimiento): las tres corridas de benchmark representan un escenario "caché frío" uniforme, sin comparación posible contra un escenario "caché caliente".

Fuga menor de datos pendiente de corrección. Las respuestas de la API que incluyen el objeto Usuario devuelven también el campo de hash de contraseña (BCrypt). Aunque el valor no es explotable directamente (es un hash, no la contraseña en texto plano), es una fuga de información innecesaria; la corrección recomendada es introducir un DTO de respuesta que excluya ese campo. Identificado durante esta entrega, pendiente de aplicar.

3. Arquitectura actual

Los diagramas C4 (niveles 1 a 3) están versionados como código en Structurizr DSL bajo docs/arquitectura/ y se exportan a imagen mediante structurizr-cli. Durante la exportación de esta entrega se detectó y corrigió un error real de sintaxis en el DSL del Nivel 3 (los contenedores de base de datos y caché estaban declarados fuera del bloque softwareSystem al que pertenecen, lo cual impedía que el archivo se exportara o validara); el archivo corregido debe reemplazar al original en el repositorio.

C4 Nivel 1 — Contexto del Asistente Virtual Académico
Muestra el sistema completo, sus usuarios humanos y la única dependencia externa (Gemini API).

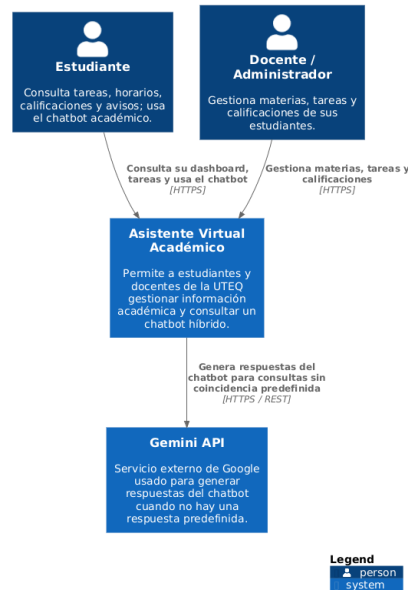


Figura 1 — C4 Nivel 1: Contexto del sistema

C4 Nivel 2 — Contenedores del Asistente Virtual Académico
Muestra los 4 contenedores desplegados (frontend, backend, PostgreSQL, Redis) y la dependencia externa de Gemini API.

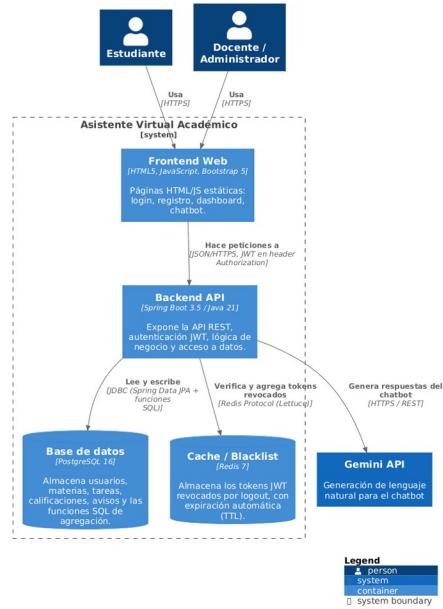


Figura 2 — C4 Nivel 2: Contenedores

C4 Nivel 3 — Componentes del Backend

Detalle interno del contenedor Backend API: controladores, filtro JWT, servicios y repositorios.

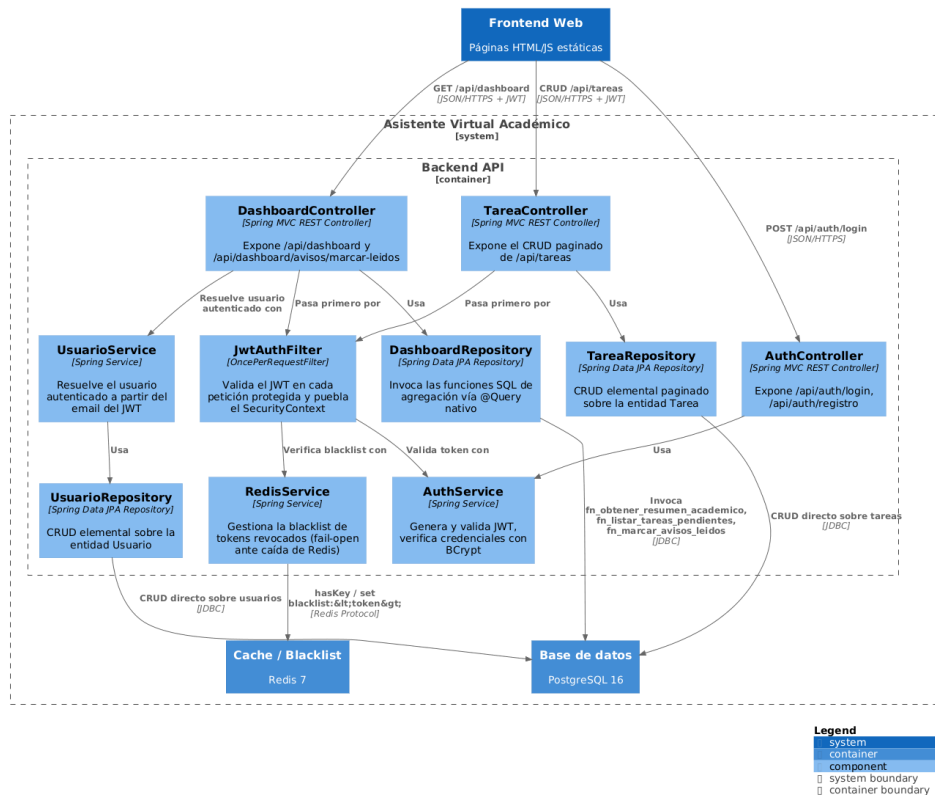


Figura 3 — C4 Nivel 3: Componentes del backend

3.1 Decisiones de arquitectura (ADR)

Los seis ADR siguen la plantilla de Nygard (Title, Status, Context, Decision, Consequences) y están versionados en docs/adr/. A continuación se resume la decisión y sus principales consecuencias (positivas y negativas) para cada uno.

ADR-001 — Elección de la pila tecnológica principal

Decisión: Spring Boot 3.5 sobre Java 21 LTS para el backend, PostgreSQL 16 como motor relacional y Redis 7 como almacén de caché/blacklist. Se descartaron Node.js/Express/Sequelize (menor soporte nativo de procedimientos almacenados parametrizados) y Django (menor experiencia previa del equipo).

Consecuencias: integración madura con procedimientos almacenados vía Jakarta Persistence 2.1 y soporte a largo plazo de Java 21 LTS, a cambio de mayor verbosidad de código y tiempos de arranque más lentos que alternativas más ligeras.

ADR-002 — Esquema de autenticación

Decisión: JWT firmado con HS256 (claims sub, rol, nombre, iat, exp), validado por un filtro JwtAuthFilter en cada petición, con revocación por logout respaldada en una blacklist de Redis con TTL igual al tiempo de expiración restante del token.

Consecuencias: escalabilidad horizontal sin estado de sesión compartido y revocación efectiva en logout, a cambio de una dependencia externa (Redis) mitigada con una política fail-open documentada. El propio ADR reconoce explícitamente, como deuda técnica, que el JWT actual no incluye aún los claims aud ni jti exigidos por una

implementación completa de RFC 7519, y que el token viaja como Bearer en el cuerpo/cabecera en vez de como cookie HttpOnly (ver limitación de la sección 2.2).

ADR-003 — Gestor de base de datos

Decisión: PostgreSQL 16, descartando MySQL/MariaDB (soporte más limitado de funciones con RETURNS TABLE) y SQL Server (licenciamiento comercial incompatible con reproducibilidad abierta).

Consecuencias: funciones SQL con proyecciones que no corresponden a ninguna entidad JPA (necesarias para el dashboard) y cero costo de licenciamiento, a cambio de una curva de aprendizaje en PL/pgSQL. Durante el desarrollo se detectó y corrigió un problema real de duplicidad de tablas por sensibilidad a mayúsculas/minúsculas entre Hibernate y PostgreSQL, resuelto estandarizando todo el esquema a minúsculas sin comillas.

ADR-004 — Estrategia de caché

Decisión: Redis 7 usado exclusivamente para la blacklist de tokens JWT revocados por logout, con TTL igual a la expiración restante de cada token.

Consecuencias: verificación de blacklist en tiempo $O(1)$ y limpieza automática sin job aparte, a cambio de un componente de infraestructura adicional. El propio ADR documenta explícitamente, como deuda técnica reconocida desde su redacción, que Redis no se usa todavía para cachear respuestas de lectura frecuente (por ejemplo, el dashboard), quedando esa mejora pendiente para la Entrega Final — este es el mismo punto documentado como limitación en la sección 2.2 de este informe.

ADR-005 — Estrategia de despliegue

Decisión: Docker Compose con 4 servicios (postgres, redis, backend, frontend), cada imagen base fijada por digest sha256 en vez de por tag variable, esquema y datos semilla aplicados exclusivamente vía docker-entrypoint-initdb.d, y un Makefile con los objetivos up/down/test/bench/audit/clean como interfaz única.

Consecuencias: reproducibilidad verificada de punta a punta (clonación limpia → make up → sistema funcional, sin pasos manuales, confirmado durante esta entrega), a cambio de que los digests deban actualizarse manualmente cuando se requiera una versión más reciente de alguna imagen base. El despliegue no incluye todavía un ambiente de producción públicamente accesible, exigido recién en la Entrega Final.

ADR-006 — Estrategia de acceso a datos

Decisión: separación estricta entre CRUD elemental (Spring Data JPA/ORM) y toda operación con JOIN, agregación, actualización masiva o proyecciones ajenas a una entidad JPA, encapsulada en funciones PL/pgSQL versionadas bajo db/procs/, invocadas vía @Procedure o @NamedStoredProcedureQuery conforme a Jakarta Persistence 2.1.

Consecuencias: cumplimiento directo del Bloque A.2 de la guía y catálogo auditable de procedimientos (docs/basedatos/CATALOGO-SP.md) independiente del código Java, a cambio de un punto adicional de coordinación entre el esquema SQL versionado y las entidades JPA.

3.2 Atributos de calidad (ISO/IEC 25010)

Característica	Prioridad	Estrategia adoptada
Adecuación funcional	Must	Resumen académico resuelto en una sola petición vía fn_obtener_resumen_academico, en vez de múltiples llamadas desde el frontend
Eficiencia de desempeño	Must	Agregación resuelta en la base de datos (PL/pgSQL); verificado empíricamente en el Bloque C.1 (p95 < 5ms)
Compatibilidad	Should	API REST con contratos JSON estándar, sin acoplamiento al frontend HTML propio
Usabilidad	Should	Dashboard con tarjetas que replican las secciones mentales del usuario (horarios, tareas, calificaciones, avisos)

Característica	Prioridad	Estrategia adoptada
Fiabilidad (tolerancia a fallos)	Must	RedisService usa política fail-open: una caída de Redis no bloquea la autenticación de tokens válidos
Fiabilidad (capacidad de recuperación)	Must	Docker Compose con imágenes por digest sha256; make up verificado de punta a punta en esta entrega
Seguridad (confidencialidad)	Must	JwtAuthFilter valida firma y expiración en cada petición antes de llegar al controlador
Seguridad (resistencia a inyección)	Must	Separación estricta ORM/procedimientos, parámetros nombrados en toda consulta no elemental
Seguridad (control de acceso)	Must	El id_usuario se resuelve siempre desde el JWT autenticado, nunca desde un parámetro del cliente (corregido de un hallazgo real, ver sección 6.2)
Mantenibilidad (modularidad)	Should	Cada función SQL en un archivo versionado independiente bajo db/procs/, documentada en CATALOGO-SP.md
Mantenibilidad (capacidad de prueba)	Should	Migraciones versionadas con Flyway; nunca ddl-auto=update
Portabilidad (adaptabilidad)	Could	Todo el stack orquestado con Docker Compose; único requisito del host es tener Docker

4. Matriz de trazabilidad

La matriz completa (docs/trazabilidad/matriz.csv) traza cada requisito hacia su historia de usuario, caso de uso, módulo de código, endpoint, prueba automatizada, tipo de acceso a datos (CRUD-ORM o procedimiento almacenado) y evidencia empírica. Un script de validación (scripts/validate-traceability.sh), integrado en make audit, verifica automáticamente que ningún requisito carezca de trazabilidad mínima.

De los 8 requisitos funcionales/no funcionales con prioridad Must, 3 están en estado verificado (con evidencia empírica cerrada), 3 en estado implementado (código y prueba manual, evidencia empírica en curso) y 2 en estado pendiente (REQ-F-006, el chatbot, y REQ-NF-001, el benchmark de rendimiento del dashboard específico — distinto del benchmark general ya cerrado sobre /api/tareas). Los 2 requisitos Should (REQ-F-005, REQ-NF-003) están ambos en estado implementado/verificado. La matriz cubre el 100% de los requisitos Must vigentes, conforme exige el Bloque A.3.3.

ID	Tipo	Prioridad	Módulo	Tipo acceso	Estado
REQ-F-001	Funcional	Must	AuthController; AuthService	CRUD-ORM	verificado
REQ-F-002	Funcional	Must	JwtAuthFilter	CRUD-ORM	implementado
REQ-F-003	Funcional	Must	TareaController; TareaRepository	CRUD-ORM	implementado
REQ-F-004	Funcional	Must	DashboardController; DashboardRepository	SP	implementado
REQ-F-005	Funcional	Should	DashboardController; DashboardRepository	SP	implementado
REQ-F-006	Funcional	Must	(sin implementar — chatbot)	—	pendiente
REQ-NF-001	No funcional	Must	DashboardController	SP	pendiente
REQ-NF-002	No funcional	Must	fn_obtener_resumen_academico; fn_listar_tareas_pendientes; fn_marcar_avisos_leidos	SP	verificado
REQ-NF-003	No funcional	Should	RedisService	CRUD-ORM	verificado
REQ-NF-004	No funcional	Must	docker-compose.yml; Makefile; db/schema.sql	—	verificado

5. Protocolo experimental

5.1 Rendimiento (k6)

Herramienta: k6 v2.1.0. Configuración: 50 VUs, con ramp-up de 10s, carga sostenida de 30s y ramp-down de 5s (k6/opts.js). Tres corridas independientes contra GET /api/tareas?page=0&size=10, autenticando una vez por corrida (setup()) con el usuario semilla (admin@uteq.edu.ec) y reutilizando el token JWT vía cabecera Authorization: Bearer en cada iteración. Certificado autofirmado (backend en https://localhost:8443), por lo que se usó insecureSkipTLSVerify.

5.2 Seguridad (OWASP)

Auditoría manual con curl contra los 6 controles mínimos exigidos (A01, A02, A03, A05, A07, A09), con la salida completa de cada comando archivada en docs/mediciones/sec/. El control A01 se verificó contra el propio hallazgo corregido durante esta entrega (aislamiento de tareas entre usuarios).

5.3 Cobertura (JaCoCo)

mvn clean test ejecutado localmente contra una instancia real de PostgreSQL (no una base en memoria), con el agente JaCoCo 0.8.12 adjunto vía Maven. 23 pruebas JUnit 5 + MockMvc, 0 fallos, 0 errores. El reporte HTML/XML/CSV se genera directamente en docs/mediciones/jacoco/. Nota metodológica: dado que la imagen Docker del backend se construye con mvn package -DskipTests (ver Dockerfile, Bloque B.1, para separar el build de producción de la ejecución de pruebas), este reporte se regenera ejecutando las pruebas de forma local, no dentro del contenedor.

5.4 Accesibilidad y calidad web (Lighthouse)

Lighthouse CLI 13.4.1 (vía @lhci/cli), throttling method "simulate" con perfil de red aproximado a Slow 4G (RTT 150ms, throughput 1638.4kbps, CPU slowdown 4x), ejecutado contra http://localhost/index.html servido por el contenedor Nginx. Nota metodológica: se usó lighthouse directo en vez de lhci autorun por un defecto conocido de chrome-launcher en Windows (EPERM al limpiar la carpeta temporal del navegador tras cerrar Chrome) que interrumpe el proceso de lhci después de generar el reporte; el JSON de resultados se confirmó guardado correctamente antes del error de limpieza, por lo que la medición en sí no se vio afectada.

5.5 Usabilidad (SUS) — pendiente

No se ejecutó en este ciclo. El protocolo exige un mínimo de 10 participantes externos al equipo con consentimiento informado firmado (plantilla ya preparada en docs/etica/consentimientos/plantilla.md). Dada la restricción de tiempo entre la publicación de esta guía y la fecha de cierre, no fue posible reclutar y coordinar sesiones con participantes reales. Queda como tarea explícita para la Entrega Final.

6. Resultados por bloque

6.1 Rendimiento

Corrida	avg (ms)	p50 (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Error rate
1	3.22	3.21	3.60	3.71	4.05	0.00%
2	3.29	3.27	3.63	3.74	4.29	0.00%
3	3.49	3.41	4.01	4.21	4.93	0.00%

Umbral objetivo: p95 < 500ms (caché frío). Resultado: 3.71–4.21ms en las tres corridas, muy por debajo del umbral. Tasa de errores HTTP ≥500: 0% en las tres corridas. Como se documentó en la sección 5.1, estas mediciones representan un único escenario (sin caché activa), no una comparación caliente/frío.

6.2 Seguridad OWASP

Los 6 controles mínimos están evidenciados con salida real de curl en docs/mediciones/sec/, más las pruebas automatizadas de AuthControllerTest y TareaControllerTest que verifican el mismo comportamiento de forma reproducible en cada ejecución de la suite.

- A01 (control de acceso): un usuario autenticado que solicita una tarea de otro usuario recibe 403 Forbidden. Verificado con evidencia curl y con las pruebas usuarioBNoPuedeVerTareaDeUsuarioADevuelve403 y equivalentes para editar/eliminar.
- A02 (fallo criptográfico): backend accesible en https://localhost:8443 con certificado autofirmado, TLS activo y verificado end-to-end durante esta entrega (ver corrección del keystore y del mapeo de puerto en docker-compose.yml).
- A03 (inyección): un payload ' OR '1'='1 en el campo email de login recibe 422 con ProblemDetails, rechazado por el patrón de validación antes de llegar a cualquier consulta. Verificado con evidencia curl y con la prueba automatizada loginConPayloadDeInyeccionEnEmailDevuelve422.
- A05 (configuración de seguridad): la evidencia capturada (docs/mediciones/sec/A05-headers-seguridad.txt) confirma la presencia de X-Content-Type-Options: nosniff, X-Frame-Options: DENY y Content-Security-Policy: default-src 'self'; frame-ancestors 'none'. Hallazgo pendiente: esa captura corresponde a una respuesta por HTTP plano (puerto 8080, previo a la activación de TLS en 8443), por lo que no incluye el encabezado Strict-Transport-Security exigido por el control A05 completo; se recomienda recapturar esta evidencia contra https://localhost:8443 antes de la Entrega Final.
- A07 (identificación y autenticación): al sexto intento fallido consecutivo desde la misma IP, el sistema responde 429 Too Many Requests. Verificado con evidencia curl y con la prueba automatizada seisIntentosFallidosConsecutivosDevuelve429EnElSexto, agregada durante esta entrega.
- A09 (registro y monitoreo): cada intento de login, exitoso o fallido, se registra con IP, timestamp y el sub (email) involucrado, por ejemplo: "LOGIN FALLIDO | ip=... | sub=tarea.usuarioA@uteq.edu.ec | motivo=contrasena incorrecta | intento=1", evidencia archivada en docs/mediciones/sec/A09-registro-logs.txt.

6.3 Cobertura de pruebas

Métrica	Cobertura
Instrucciones	81.6%
Líneas	80.7%
Ramas (branches)	66.1%
Pruebas ejecutadas	23 (0 fallos, 0 errores)

Supera ampliamente el mínimo de 60% exigido para esta entrega, y ya alcanza el objetivo de 70% fijado para la Entrega Final.

6.4 Accesibilidad y calidad web (Lighthouse)

Categoría	Umbral mínimo	Resultado
Performance	≥ 80	91
Accessibility	≥ 90	100
Best Practices	≥ 90	96
SEO	≥ 90	90

La primera corrida detectó dos hallazgos reales de accesibilidad en index.html (contraste de color insuficiente en el texto secundario y en el acento dorado del logo, y ausencia de un landmark <main>), ambos corregidos durante esta entrega; la re-auditoría confirmó Accessibility 100/100.

6.5 Usabilidad (SUS)

Pendiente — ver sección 5.5. No se reporta puntuación SUS en esta entrega.

7. Amenazas a la validez

- El benchmark de rendimiento se ejecutó con el generador de carga (k6) y el sistema bajo prueba en la misma máquina física, sin latencia de red real; los tiempos de respuesta reportados no son representativos de un despliegue en producción con cliente y servidor separados.
- No existe comparación caché caliente/frío porque el sistema, en su estado actual, no implementa una capa de caché de respuestas HTTP (ver limitación documentada en la sección 2.2).
- La auditoría de Lighthouse se ejecutó en una sola corrida (no las tres corridas independientes recomendadas para reducir varianza de medición), por una limitación de tiempo, no metodológica.
- No hay evidencia de usabilidad con usuarios reales (SUS) en este ciclo; toda afirmación sobre la facilidad de uso del sistema es, por ahora, cualitativa y no está respaldada empíricamente.
- La cobertura de pruebas se midió ejecutando la suite localmente contra una instancia de PostgreSQL en el host, no contra el contenedor Docker desplegado; se asume paridad de comportamiento entre ambos entornos, pero no se verificó de forma independiente.

8. Declaración de contribuciones (CRediT)

Fajardo Montes Michael Xavier — Conceptualization, Software (backend, frontend, Docker/reproducibilidad), Investigation (diagnóstico y corrección de defectos), Writing – original draft (SRS, historias de usuario, bitácora de observaciones, este informe), Project administration.

Rivera Suárez Jonathan Javier — Documentation (los 6 ADR), Software (diagramas C4 en Structurizr DSL), Writing – original draft (tabla ISO/IEC 25010); asume tareas adicionales tras la baja de Castro Palma.

Castro Palma Justyn Moisés — Se dio de baja de la materia durante el desarrollo del proyecto. Contribuyó a las Entregas 1A/1B y a los diagramas iniciales del equipo antes de su retiro.

9. Declaración ética

Todos los datos utilizados durante el desarrollo (usuarios, materias, tareas, calificaciones, avisos) son datos sintéticos de prueba creados por el propio equipo; no se ha usado, en ningún momento, una base de datos real de estudiantes de la UTEQ ni de ninguna otra institución. El sistema, en su estado v0.9.0-rc, no gestiona datos de estudiantes reales; los campos de información personal identificable están poblados únicamente con datos ficticios en todo el desarrollo y las pruebas. Las contraseñas se almacenan con hash BCrypt. Para las pruebas de usabilidad (Bloque C.3, pendientes), el equipo cuenta con una plantilla de consentimiento informado (docs/etica/consentimientos/plantilla.md) que cada participante deberá firmar antes de participar; los formularios firmados no se suben al repositorio público. Declaración completa en docs/etica/ETHICS.md.

10. Próximos pasos hacia la Entrega Final

La Tercera Entrega deja al equipo en el estado v0.9.0-rc, candidato a versión estable. La transición a v1.0.0 debe absorber la evaluación de esta entrega y, en particular, atender los siguientes puntos ya identificados durante este ciclo:

- Migrar el JWT de Bearer token a cookie HttpOnly+Secure+SameSite=Strict, o documentar formalmente en el ADR-002 la decisión de mantener Bearer token con su justificación.
- Implementar caché de respuestas HTTP con Redis (TTL externo, hit ratio medido) en el endpoint de listado, tal como reconoce el propio ADR-004 como deuda técnica pendiente.
- Añadir los claims aud y jti al JWT, conforme a RFC 7519, según lo anota el ADR-002.
- Introducir un DTO de respuesta para Usuario que excluya el campo de hash de contraseña.
- Ejecutar la prueba de usabilidad SUS con al menos 10 participantes reales, usando la plantilla de consentimiento ya preparada.

- Recapturar la evidencia del control OWASP A05 contra el endpoint HTTPS (puerto 8443) para incluir el encabezado Strict-Transport-Security.
- Exportar los diagramas C4 (L1–L3) de Structurizr DSL a PNG e incorporarlos a este informe.
- Elevar la cobertura de pruebas del 80.7% actual hacia el 70% ya superado, ampliando la cobertura de ramas (66.1% actual) en los controladores.
- Completar el chatbot (HU-04 / CU-04 / REQ-F-006), única funcionalidad Must sin implementar en el estado actual.

Referencias

International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.

ISO/IEC 25010:2011 — Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE).

Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT). RFC 7519, IETF, 2015.

Nottingham, M., Wilde, E. Problem details for HTTP APIs. RFC 7807, IETF, 2016.

OWASP Foundation. OWASP Top 10:2021 — The ten most critical web application security risks, 2021.

Nygard, M. Documenting architecture decisions. Cognitect Blog, 2011.

Brooke, J. SUS: A 'quick and dirty' usability scale. En Usability Evaluation in Industry, Taylor and Francis, 1996.

National Information Standards Organization (NISO). ANSI/NISO Z39.104-2022, CRediT, Contributor Roles Taxonomy, 2022.

Preston-Werner, T. Semantic Versioning 2.0.0, 2013.

Wilkinson, M. D. et al. The FAIR guiding principles for scientific data management and stewardship. Scientific Data, 3:160018, 2016.

Association for Computing Machinery. Artifact review and badging — version 1.1, 2020.